Amazon - 100 Technical Interview Questions

Amazon (SDE-1, New Grad)

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. What is the difference between composition and inheritance? Which is generally preferred and why?

Answer:

Composition means a class holds a reference to another class as a field ('has-a', e.g. a Car has an Engine), while inheritance means a class extends another ('is-a', e.g. a Car is-a Vehicle). Composition is generally preferred because it's more flexible (behaviour can be swapped at runtime) and avoids the tight coupling and fragile-base-class problems that deep inheritance hierarchies create - this is the idea behind 'favor composition over inheritance'.

Q2. What is the difference between method overloading and method overriding? Give a code example of each.

Answer:

Overloading means multiple methods with the same name but different parameter lists in the same class (resolved at compile time), e.g. add(int,int) and add(double,double). Overriding means a subclass provides its own implementation of a method already defined in its parent with the same signature (resolved at runtime), e.g. Dog overriding Animal's makeSound().

Q3. What is polymorphism? Differentiate between compile-time and runtime polymorphism with examples.

Answer:

Polymorphism lets the same interface behave differently depending on the actual object. Compile-time (static) polymorphism is resolved during compilation - method/operator overloading is the classic example. Runtime (dynamic) polymorphism is resolved during execution based on the actual object type - method overriding via a base-class reference is the classic example.

Q4. What is constructor overloading? Can constructors be inherited?

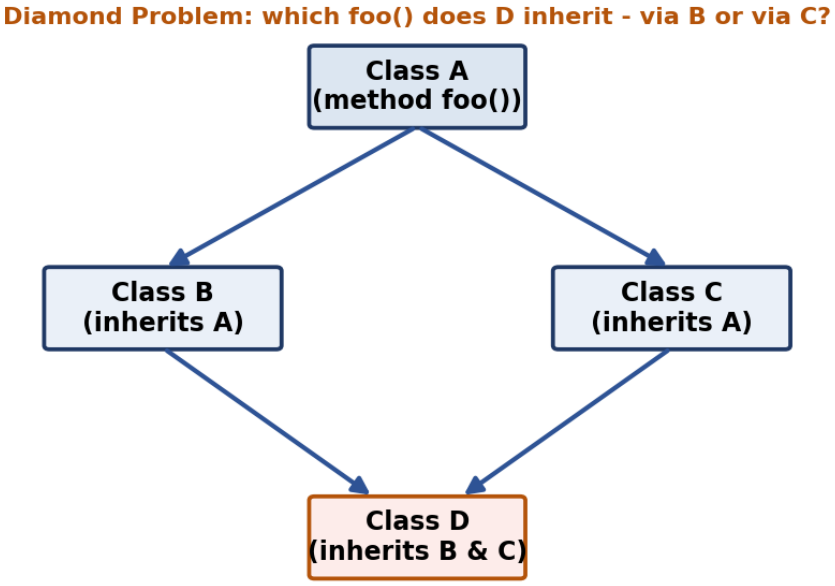
Answer:

Constructor overloading means a class has multiple constructors with different parameter lists, letting objects be created in different ways (e.g. Point() vs Point(x,y)). Constructors are NOT inherited by subclasses - a subclass must define its own constructors, though it can call a parent constructor explicitly via super().

Q5. What is the 'diamond problem' in multiple inheritance, and how does Java avoid it?

Answer:

The diamond problem arises when a class inherits from two classes that both inherit from a common base class, creating ambiguity about which inherited method version to use. Java avoids it for classes by disallowing multiple inheritance of classes entirely (a class can extend only one class), and for interfaces (Java 8+ default methods) it forces the implementing class to explicitly override the conflicting method if two interfaces provide the same default.



Q6. Explain encapsulation and why it is important for maintainable software.

Answer:

Encapsulation hides an object's internal state behind private fields and exposes controlled access through public getters/setters or methods. It matters because it lets you change internal implementation without breaking external code, enforce validation (e.g. reject a negative balance), and reduces unintended coupling between modules.

Q7. What are access modifiers? Explain public, private, protected, and default with examples.

Answer:

Access modifiers control visibility: 'public' - accessible from anywhere; 'private' - accessible only within the same class; 'protected' - accessible within the same package and by subclasses (even in different packages, in Java); 'default' (no modifier) - accessible only within the same package. They enforce encapsulation by controlling what other code can touch.

Q8. Explain the concept of a virtual function in C++ and how it enables runtime polymorphism.

Answer:

A virtual function in C++ is declared with the 'virtual' keyword in a base class, telling the compiler to resolve the call at runtime via a vtable based on the actual object type rather than the reference/pointer type. This is what allows a Shape* pointing to a Circle to correctly call Circle's draw() instead of Shape's - the mechanism behind runtime polymorphism in C++.

Q9. Explain what a static method is and why it cannot be overridden (only hidden) in Java.

Answer:

A static method belongs to the class itself rather than any instance, and is resolved at compile time based on the reference type, not the runtime object type. Since overriding relies on runtime (dynamic) dispatch, a static method with the same signature in a subclass merely 'hides' the parent's version rather than overriding it - the version called depends on which class name/reference type you use to call it.

Q10. Explain the difference between an abstract class and an interface. When would you choose one over the other?

Answer:

An abstract class can mix abstract and concrete methods, hold state (fields), and a class can extend only one abstract class. An interface (pre-Java 8) only declares method signatures with no implementation, and a class can implement many interfaces. Use an abstract class when subclasses share common state/behaviour; use an interface when you only need to guarantee a contract across unrelated classes.

DBMS & SQL

Q11. Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

SELECT e.name, e.salary, e.department FROM Employee e WHERE e.salary > (SELECT AVG(salary) FROM Employee e2 WHERE e2.department = e.department). The correlated subquery recomputes the department's average salary for each row and compares it to that employee's own salary.

Q12. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age >= 18), independent of any other table.

Q13. What is sharding, and how does it help scale a database horizontally?

Answer:

Sharding splits a large table's rows horizontally across multiple database servers (shards), typically based on a shard key (e.g. customer_id range or hash), so no single machine needs to store or query the entire dataset. It improves write and storage scalability but introduces challenges like cross-shard joins/transactions and rebalancing when adding new shards.

Q14. How would you optimize a slow-running SQL query? Walk through your debugging approach.

Answer:

Approach: first run EXPLAIN/EXPLAIN ANALYZE to see the query plan and spot full table scans; check whether the filtered/joined columns have appropriate indexes; look for functions applied to indexed columns (which can prevent index use); check for unnecessary SELECT * pulling extra columns; consider rewriting correlated subqueries as joins; and check table statistics are up to date so the optimizer picks a good plan.

Q15. Explain the difference between a correlated subquery and a regular subquery.

Answer:

A regular (uncorrelated) subquery executes once, independently of the outer query, and its result is used by the outer query. A correlated subquery references a column from the outer query and is conceptually re-evaluated for each row processed by the outer query, e.g. finding employees who earn more than the average salary of their own department.

Q16. What is the difference between OLTP and OLAP systems?

Answer:

OLTP (Online Transaction Processing) systems handle many short, frequent read/write transactions (e.g. an ATM system) and are optimized for fast inserts/updates on normalized schemas. OLAP (Online Analytical Processing) systems handle complex, read-heavy analytical queries over large historical datasets (e.g. a sales dashboard) and are optimized with denormalized/star schemas for fast aggregation.

Q17. Explain optimistic vs pessimistic locking in database transactions.

Answer:

Pessimistic locking acquires a lock on a row before reading/modifying it, blocking other transactions until it's released - safe under high contention but can hurt throughput. Optimistic locking assumes conflicts are rare: it reads data without locking, then checks (e.g. via a version number) at commit time whether the data changed since it was read, retrying if a conflict is detected - better throughput when conflicts are infrequent.

Q18. What is a trigger, and give an example of when you would use one.

Answer:

A trigger is a stored procedure automatically executed in response to an INSERT, UPDATE, or DELETE event on a table. Example: a trigger that automatically logs the old and new values of a Salary column into an AuditLog table whenever an Employee row is updated, without requiring the application code to do it explicitly.

Operating Systems

Q19. What is thrashing, and what causes it?

Answer:

Thrashing happens when a system is so overcommitted on memory that it spends most of its time swapping pages in and out of disk rather than executing actual process instructions, causing CPU utilization and throughput to collapse even though the system appears busy. It's typically caused by too many processes running with too little physical memory, so each one's working set doesn't fit in RAM.

Q20. What is a race condition, and how can synchronization primitives like semaphores or mutexes prevent it?

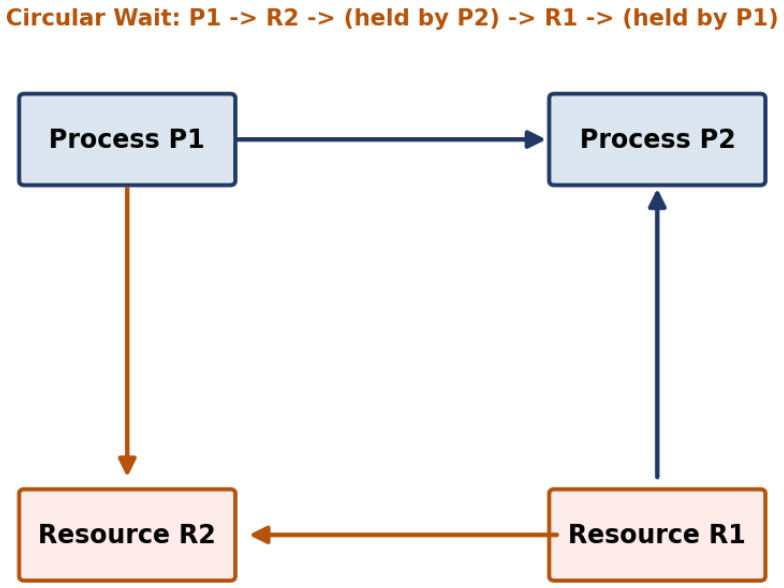
Answer:

A race condition occurs when multiple threads/processes access shared data concurrently and the final result depends on the unpredictable timing/order of execution, potentially producing incorrect results. Synchronization primitives like mutexes (mutual exclusion locks) or semaphores prevent this by ensuring only one thread can enter a critical section that modifies shared data at a time.

Q21. Explain the four necessary conditions for a deadlock to occur.

Answer:

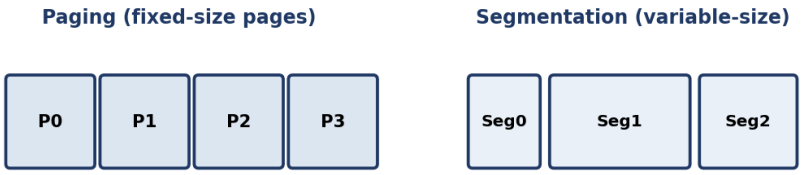
The four necessary conditions are: **Mutual Exclusion** - at least one resource is held in a non-shareable mode; **Hold and Wait** - a process holds one resource while waiting for another; **No Preemption** - resources can't be forcibly taken away, only released voluntarily; **Circular Wait** - a closed chain of processes each waiting for a resource held by the next. All four must hold simultaneously for a deadlock to occur.



Q22. Explain the difference between paging and segmentation as memory management techniques.

Answer:

Paging divides both physical and logical memory into fixed-size blocks (pages/frames), eliminating external fragmentation but potentially causing internal fragmentation within a page. Segmentation divides a program into variable-sized logical units (code, stack, heap segments) that map more naturally to how programmers think about memory, but can suffer external fragmentation since segments vary in size.



Q23. Explain the producer-consumer problem and how you would solve it using semaphores.

Answer:

The producer-consumer problem involves a producer adding items to a shared bounded buffer and a consumer removing them, needing synchronization so the producer doesn't add to a full buffer and the consumer doesn't remove from an empty one. It's solved using two counting semaphores (one tracking empty slots, one tracking filled slots) plus a mutex protecting the buffer itself during actual insert/remove operations.

Q24. What is virtual memory, and why is it useful even when physical RAM is limited?

Answer:

Virtual memory gives each process the illusion of a large, contiguous address space by mapping logical addresses to physical RAM or disk (swap space) via page tables, transparently to the program. It's useful even with limited RAM because it allows programs larger than physical memory to run, isolates each process's memory from others for safety, and lets the OS overcommit memory across many processes that don't all need their full memory at once.

Q25. What is the Banker's Algorithm, and how does it help avoid deadlocks?

Answer:

The Banker's Algorithm avoids deadlock by only granting a resource request if the resulting state is 'safe' - meaning there exists some order in which all processes could still finish given the maximum resources they might request. It simulates allocation before actually granting it, refusing requests that could lead to an unsafe (potentially deadlocked) state.

Q26. Explain the difference between preemptive and non-preemptive scheduling with examples of each.

Answer:

Preemptive scheduling allows the OS to forcibly suspend a running process to give the CPU to another (e.g. Round Robin, Priority Scheduling with preemption) - responsive but has context-switch overhead. Non-preemptive scheduling lets a running process keep the CPU until it voluntarily yields or finishes (e.g. FCFS, non-preemptive SJF) - simpler but a long process can block shorter ones ('convoy effect').

Computer Networks

Q27. Explain congestion control mechanisms used by TCP (e.g., slow start, congestion avoidance).

Answer:

TCP congestion control prevents a sender from overwhelming the network. 'Slow start' begins with a small congestion window and doubles it each round-trip until reaching a threshold or detecting loss. 'Congestion avoidance' then increases the window more conservatively (linearly) to probe for more bandwidth without causing congestion. On packet loss, TCP reduces the window sharply (e.g. halving it) and restarts the process, adapting to network conditions.

Q28. What is ARP, and how does it map an IP address to a MAC address?

Answer:

ARP (Address Resolution Protocol) resolves a known IP address to its corresponding MAC address on a local network. When a device needs to send a frame to an IP it doesn't have a MAC address for, it broadcasts an ARP request ('who has this IP?'), and the device owning that IP responds with its MAC address, which the sender then caches for future use.

Q29. Explain the difference between unicast, multicast, and broadcast communication.

Answer:

Unicast sends data from one sender to exactly one specific receiver (e.g. a normal web request). Multicast sends data from one sender to a specific group of interested receivers simultaneously (e.g. IPTV streaming to subscribed clients). Broadcast sends data from one sender to every device on the local network segment, regardless of interest (e.g. ARP requests, DHCP discovery).

Q30. What is a CDN, and how does it reduce latency for end users?

Answer:

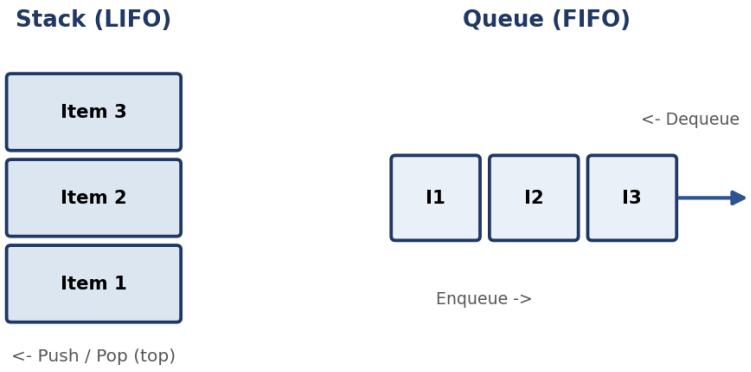
A CDN (Content Delivery Network) is a geographically distributed network of cache servers that store copies of static content (images, videos, scripts) closer to end users. When a user requests content, it's served from the nearest CDN edge server rather than the origin server, reducing round-trip latency and offloading traffic from the origin.

Data Structures & Algorithms

Q31. What is the difference between a stack and a queue, and where is each used in real systems?

Answer:

A stack follows Last-In-First-Out (LIFO) order - used for undo functionality, expression evaluation, and the call stack behind recursion. A queue follows First-In-First-Out (FIFO) order - used for task scheduling, breadth-first search, and handling requests in the order they arrive (e.g. print queues, message queues).



Q32. What is memoization, and how does it improve the performance of recursive algorithms?

Answer:

Memoization stores the result of a function call keyed by its input parameters (often in a hashmap or array), so if the same inputs occur again, the cached result is returned instantly instead of recomputing it. This turns algorithms with exponential-time naive recursion (like plain recursive Fibonacci) into polynomial-time algorithms by eliminating redundant recursive calls on the same subproblem.

Q33. Explain how a trie data structure works and where it is useful (e.g., autocomplete).

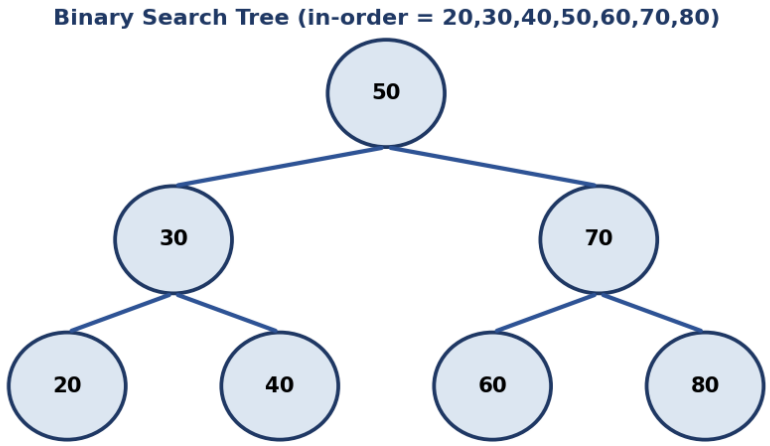
Answer:

A trie (prefix tree) stores strings character by character along tree edges, where each path from the root represents a prefix, and nodes are often marked to indicate a complete word ends there. It's especially useful for prefix-based operations like autocomplete/typeahead suggestions, spell-checking, and IP routing tables, since searching or inserting a string takes time proportional to the string's length rather than the number of stored strings.

Q34. What is a binary search tree, and what is the time complexity of insertion, deletion, and search in the average and worst case?

Answer:

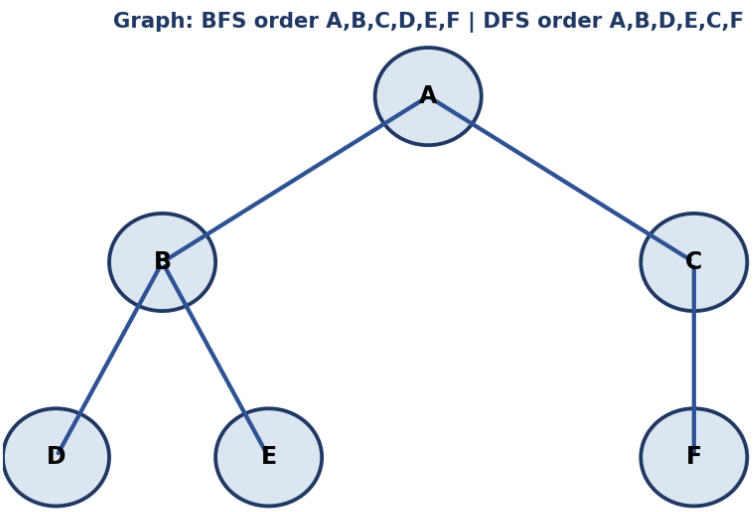
A Binary Search Tree (BST) is a tree where every node's left subtree contains only smaller values and its right subtree contains only larger values, enabling efficient ordered operations. In a balanced BST, insertion, deletion, and search are all O(log h) where h is the height (roughly log n for a balanced tree). In the worst case (a degenerate, list-like tree caused by inserting sorted data without rebalancing), all three operations degrade to O(n).



Q35. Explain the difference between BFS and DFS graph traversal, including their use cases.

Answer:

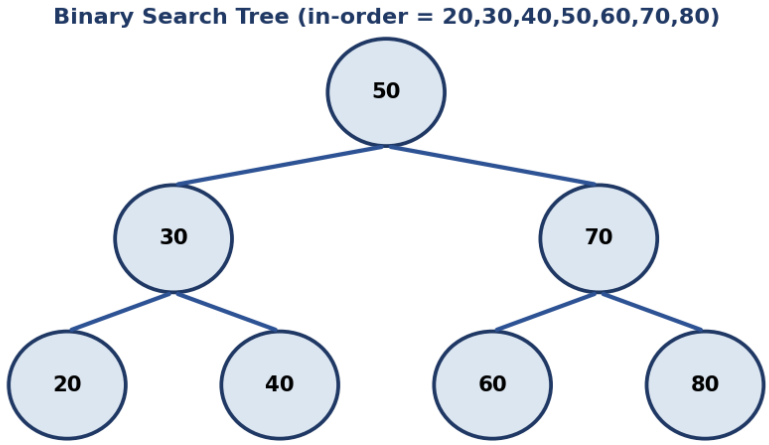
BFS (Breadth-First Search) explores a graph level by level using a queue, visiting all neighbors of a node before moving deeper - useful for finding the shortest path in an unweighted graph. DFS (Depth-First Search) explores as far as possible along one branch before backtracking, using a stack (or recursion) - useful for tasks like detecting cycles, topological sorting, or exploring all possible paths/states.



Q36. What is the difference between a balanced and an unbalanced binary search tree, and why does balance matter?

Answer:

A balanced BST keeps its height close to log n by rebalancing after insertions/deletions (e.g. AVL trees, Red-Black trees), guaranteeing O(log n) search/insert/delete. An unbalanced BST can degenerate into something resembling a linked list if elements are inserted in sorted order without rebalancing, degrading all operations to O(n). Balance matters because it's what actually delivers the logarithmic performance a BST is chosen for in the first place.



Q37. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in O(E log V). Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q38. What is Big-O notation, and why do we use asymptotic analysis instead of exact runtimes?

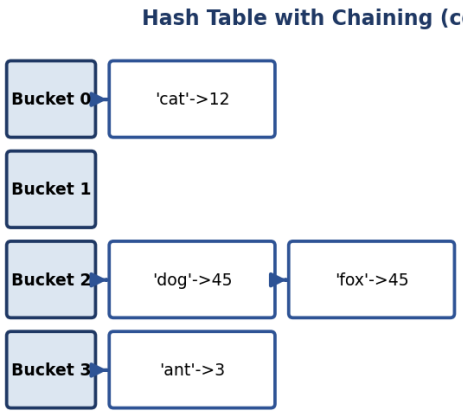
Answer:

Big-O notation describes the upper-bound growth rate of an algorithm's running time or space as input size approaches infinity, ignoring constant factors and lower-order terms. We use asymptotic analysis instead of exact runtimes because exact timings vary by hardware, language, and implementation details, while Big-O gives a hardware-independent way to compare how algorithms scale as input grows large.

Q39. Explain how a hash table works internally, including collision resolution strategies.

Answer:

A hash table maps keys to array indices using a hash function, giving average O(1) time for insert/search/delete. Since different keys can hash to the same index (a collision), collisions are typically resolved via chaining (each bucket holds a linked list/array of entries that hashed there) or open addressing (probing for the next free slot in the array itself, e.g. linear or quadratic probing).



Q40. How would you find the kth largest element in an unsorted array efficiently?

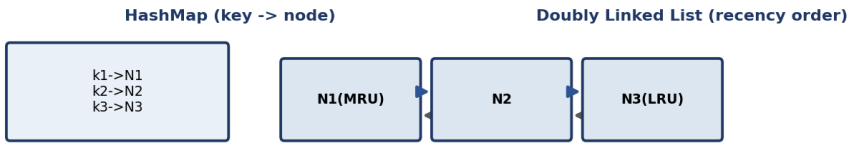
Answer:

One efficient approach uses a min-heap of size k: iterate through the array, pushing elements onto the heap and popping the smallest whenever the heap exceeds size k - after processing all elements, the heap's root is the kth largest, achieving O(n log k) time. Alternatively, Quickselect (a partition-based approach related to Quicksort) finds it in average O(n) time.

Q41. Explain how you would design an LRU cache and the data structures you'd use to achieve O(1) get/put operations.

Answer:

An LRU cache needs O(1) get and put operations. The standard design combines a hashmap (key -> pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in O(1); when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in O(1), with the hashmap updated accordingly.



O(1) get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q42. Explain how binary search works and derive its time complexity.

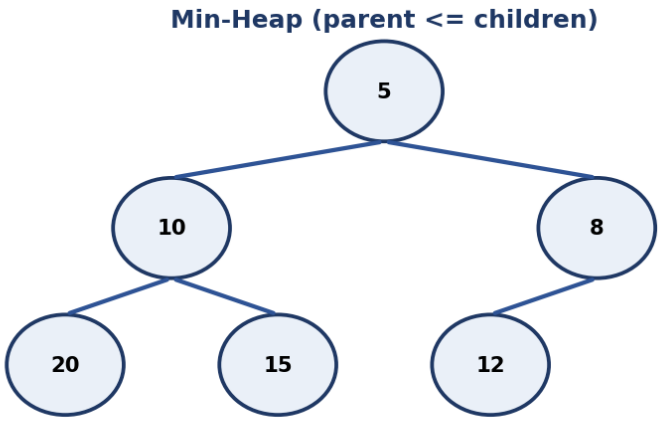
Answer:

Binary search repeatedly divides a sorted array in half: compare the target to the middle element, and discard the half that can't contain the target, narrowing the search range each time. Since the search space is halved every step, the time complexity is O(log n) - a huge improvement over linear search's O(n) for large sorted datasets.

Q43. Explain how a min-heap/max-heap is implemented and how it supports efficient priority queue operations.

Answer:

A heap is a complete binary tree (usually implemented as an array) satisfying the heap property: in a min-heap, every parent is less than or equal to its children (so the minimum is always at the root); in a max-heap, every parent is greater than or equal to its children. Insertion and extraction of the min/max both take O(log n) since you only need to 'bubble' an element up or down the height of the tree, making heaps the standard backing structure for an efficient priority queue.



Q44. What is the difference between a greedy algorithm and dynamic programming? Give an example problem for each.

Answer:

A greedy algorithm makes the locally optimal choice at each step without reconsidering past decisions, and works only for problems where local optimality leads to a global optimum (e.g. activity/interval selection, Huffman coding). Dynamic programming considers overlapping subproblems and combines their optimal solutions, guaranteeing a global optimum for a broader class of problems (e.g. the 0/1 Knapsack problem, where greedy choice-by-choice doesn't always give the best result).

Q45. Compare Merge Sort and Quick Sort - discuss best/worst case complexity, stability, and when you'd choose one over the other.

Answer:

Merge Sort has guaranteed O(n log n) time in best, average, and worst cases, is stable, but requires O(n) extra space for merging. Quick Sort also averages O(n log n) but can degrade to O(n^2) in the worst case (e.g. already-sorted input with a poor pivot choice), is typically not stable, but sorts in-place with better real-world constant factors and cache performance. Choose Merge Sort when stability or guaranteed worst-case performance matters (e.g. external sorting); choose Quick Sort for general in-memory sorting where average-case speed matters more.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No

Q46. What is the sliding window technique, and in what type of problems is it useful?

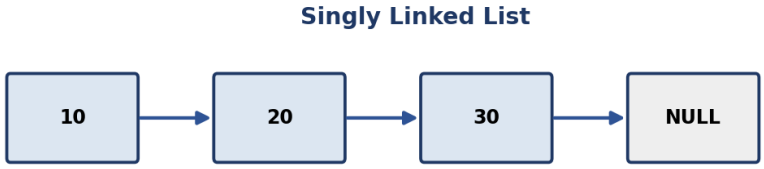
Answer:

The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k, or longest substring without repeating characters), typically turning an O(n^2) brute force into an O(n) solution.

Q47. How would you detect a cycle in a linked list? Explain Floyd's cycle detection algorithm.

Answer:

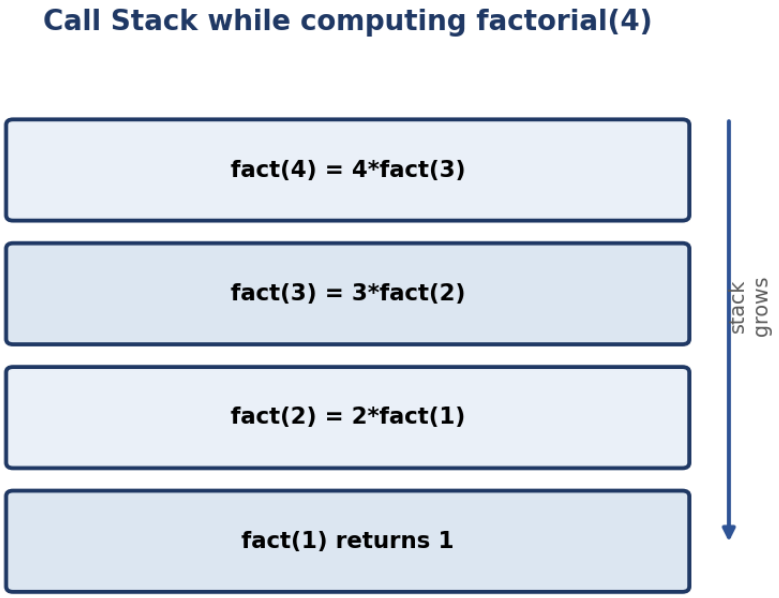
Floyd's cycle detection (the 'tortoise and hare' algorithm) uses two pointers moving through the linked list at different speeds - one advances one node at a time (slow), the other advances two nodes at a time (fast). If there is a cycle, the fast pointer will eventually 'lap' the slow pointer and they will meet at the same node; if the fast pointer reaches the end (null), there is no cycle. This runs in O(n) time and O(1) space.



Q48. What is dynamic programming, and how does it differ from plain recursion and from greedy algorithms?

Answer:

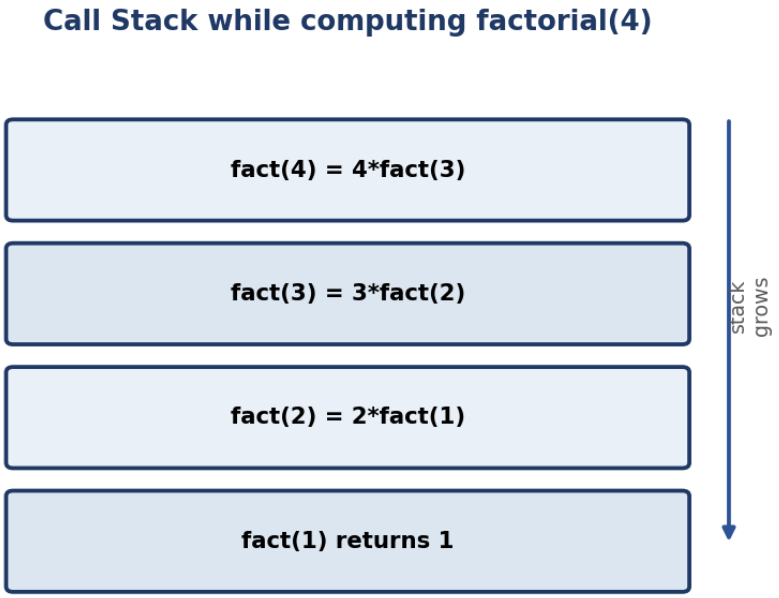
Dynamic programming solves problems by breaking them into overlapping subproblems, solving each just once, and storing (memoizing) the results to avoid redundant recomputation - it's applicable when a problem has both optimal substructure and overlapping subproblems. Plain recursion without memoization re-solves the same subproblems repeatedly, often leading to exponential time. Unlike a greedy algorithm (which makes one irrevocable locally-optimal choice at each step), DP considers combinations of subproblem solutions to guarantee a globally optimal result when applicable.



Q49. Explain how recursion uses the call stack internally, and what causes a stack overflow.

Answer:

Each recursive call pushes a new stack frame onto the call stack, containing that call's local variables, parameters, and return address. As calls nest deeper, more frames stack up; when a call returns, its frame is popped off. If recursion goes too deep (e.g. no base case, or a base case never reached due to a bug), the call stack exceeds its allocated size, causing a stack overflow.



Q50. Explain the two-pointer technique with an example problem.

Answer:

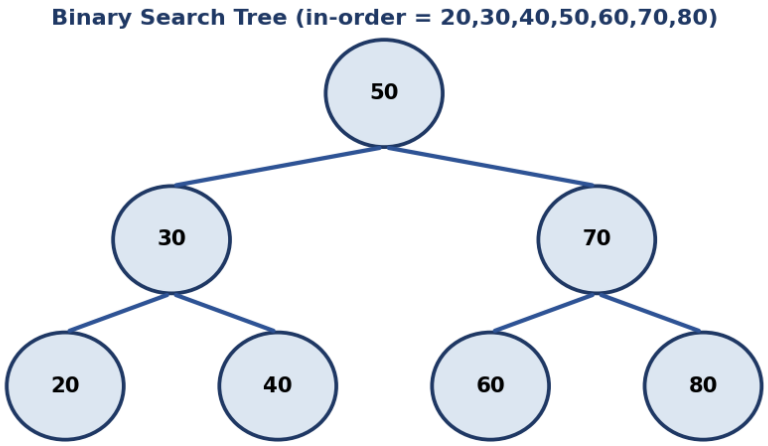
The two-pointer technique uses two indices moving through a data structure (often from opposite ends or at different speeds) to avoid nested loops. Example: given a sorted array, to find a pair summing to a target, start one pointer at the beginning and one at the end - if the sum is too large, move the right pointer left; if too small, move the left pointer right - solving the problem in O(n) instead of the O(n^2) brute force of checking every pair.

Coding Problems

Q51. Write a program to find the lowest common ancestor (LCA) of two nodes in a binary tree.

Answer:

For a BST, the LCA can be found by starting at the root and comparing both target node values to the current node's value: if both are smaller, move to the left child; if both are larger, move to the right child; if they diverge (one smaller, one larger, or one equals the current node), the current node is the LCA. This runs in O(h) time where h is the tree height. For a general binary tree (not necessarily a BST), a recursive post-order search that returns the first node where both targets are found in different subtrees is used instead.



Q52. Write a program to check whether a binary tree is balanced.

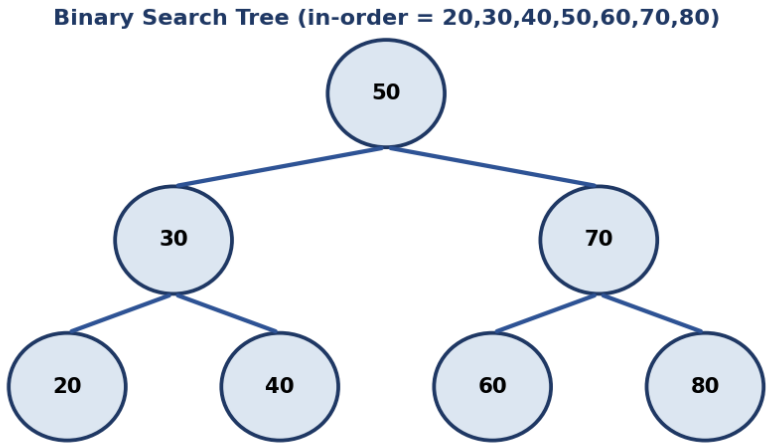
Answer:

Recursively compute the height of the left and right subtrees for every node; if at any point the difference in height between left and right subtrees exceeds 1 for any node, the tree is unbalanced. An efficient implementation computes height and checks balance in the same bottom-up traversal (returning -1 as a sentinel to signal 'unbalanced' early), achieving O(n) time instead of the naive O(n^2).

Q53. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in O(n) time.



Q54. Write a program to remove duplicates from a sorted array in-place.

Answer:

Since the array is already sorted, use a slow pointer marking the last position of a unique element written so far, and a fast pointer scanning ahead - whenever the fast pointer finds a value different from the one at the slow pointer, advance the slow pointer and copy that new value into place. This removes duplicates in-place in O(n) time and O(1) extra space.

Q55. Write a program to check whether a given string is a palindrome.

Answer:

Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in O(n) time and O(1) extra space (ignoring the input itself).

Q56. Write a program to implement quicksort or mergesort from scratch.

Answer:

Quicksort: pick a pivot element, partition the array so elements smaller than the pivot come before it and larger elements come after, then recursively sort the two partitions - average O(n log n), worst case O(n^2) with a poor pivot choice. Mergesort: recursively split the array in half until subarrays of size 1 remain, then repeatedly merge sorted subarrays back together - guaranteed O(n log n) in all cases, but needs O(n) auxiliary space for merging.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No

Q57. Write a program to find the maximum subarray sum (Kadane's Algorithm).

Answer:

Kadane's Algorithm scans the array once, maintaining a running 'current sum' that resets to the current element whenever adding the previous running sum would make it smaller than the element alone (i.e. current_sum = max(arr[i], current_sum + arr[i])), while tracking the overall maximum seen so far. This finds the maximum subarray sum in O(n) time and O(1) space.

Q58. Write a program to check if two strings are anagrams of each other.

Answer:

Two common approaches: (1) sort both strings and check if the sorted versions are equal (O(n log n)); or (2) build a frequency count (e.g. a 26-length array for lowercase letters, or a hashmap for general characters) of one string, then decrement counts while scanning the second string - if all counts return to zero and lengths match, they're anagrams (O(n) time, O(1) or O(k) space).

Q59. Write a program to flatten a nested list/array.

Answer:

Use recursion: iterate through each element of the (possibly nested) list - if an element is itself a list, recursively flatten it and extend the result with its contents; if it's a plain value, append it directly to the result list. This naturally handles arbitrary nesting depth, with time complexity O(n) where n is the total number of elements across all nesting levels.

Q60. Write a program to find the missing number in an array containing n distinct numbers from 0 to n.

Answer:

Sum formula approach: the expected sum of numbers 0 to n is n*(n+1)/2; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - O(n) time, O(1) space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q61. Write a program to find the intersection of two arrays.

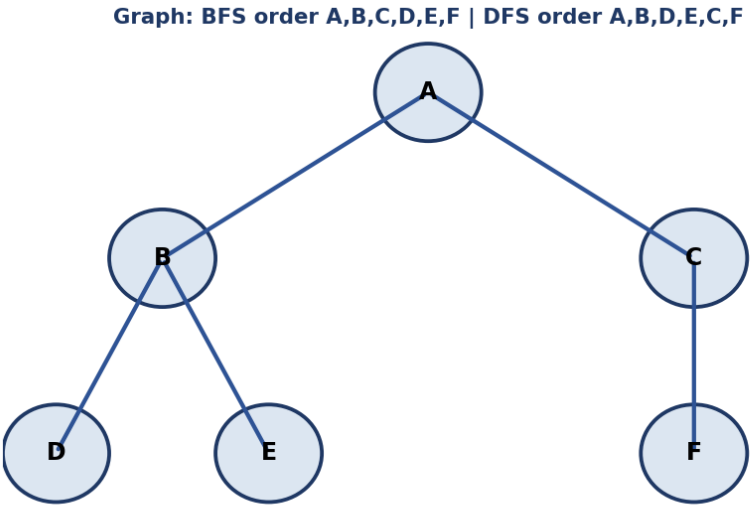
Answer:

Efficient approach: put all elements of the smaller array into a hash set for O(1) lookup, then iterate through the other array and collect any element found in the set (adding it to a result set to avoid duplicates in the output). This runs in O(n + m) time and O(min(n, m)) space.

Q62. Write a program to count the number of islands in a 2D grid (graph/matrix traversal).

Answer:

Treat the grid as an implicit graph where each land cell ('1') is a node connected to its up/down/left/right land neighbors. Scan every cell; whenever an unvisited land cell is found, increment the island count and run a BFS or DFS from that cell to mark every connected land cell as visited, so it isn't counted again. This runs in O(rows * cols) time since each cell is visited a constant number of times.



Q63. Write a program to find the first non-repeating character in a string.

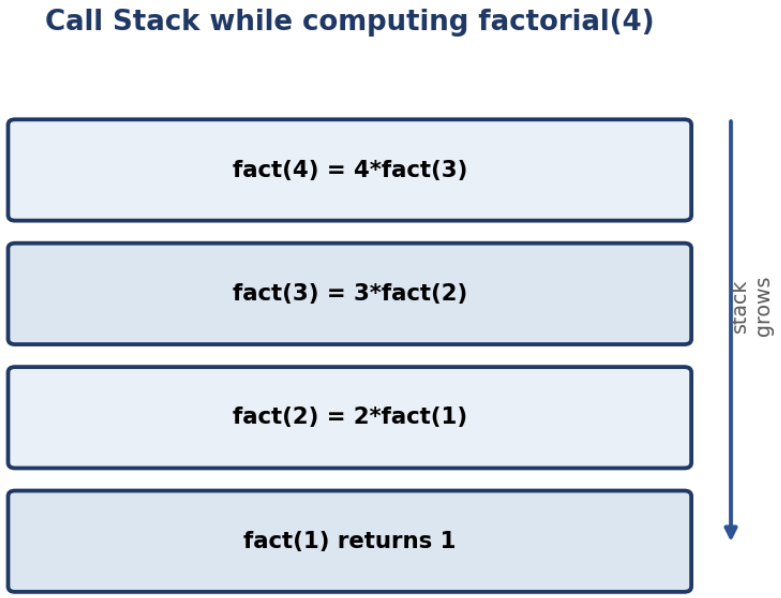
Answer:

Use a hashmap to count the frequency of each character in a single pass through the string, then iterate through the string again in order and return the first character whose count is exactly 1. This runs in O(n) time and O(k) space, where k is the size of the character set.

Q64. Write a program to find the factorial of a number using recursion.

Answer:

Recursive definition: factorial(n) = n * factorial(n-1), with a base case factorial(0) = 1 (or factorial(1) = 1). Each call multiplies n by the result of the smaller subproblem until the base case is reached, then the multiplications unwind back up the call stack. Time complexity is O(n); space complexity is O(n) due to the recursive call stack.



Q65. Write a program to find the middle element of a linked list in a single pass.

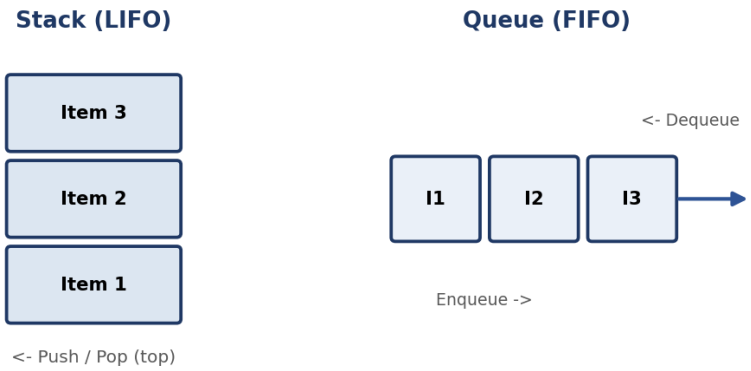
Answer:

Use the 'slow and fast pointer' technique: the slow pointer advances one node at a time while the fast pointer advances two nodes at a time - when the fast pointer reaches the end of the list, the slow pointer will be exactly at the middle. This finds the middle node in a single pass, in O(n) time and O(1) space.

Q66. Write a program to implement a stack using two queues (or vice versa).

Answer:

To implement a queue using two stacks: push new elements onto stack1; for dequeue, if stack2 is empty, pop everything from stack1 and push it onto stack2 (reversing the order), then pop from stack2 - this gives FIFO behavior with amortized O(1) per operation. Implementing a stack using two queues works similarly in reverse, rearranging elements between the two queues so the most recently added element comes out first.



Q67. Write a program to find the longest common subsequence between two strings.

Answer:

Use dynamic programming: build a 2D table dp[i][j] representing the length of the longest common subsequence between the first i characters of string A and first j characters of string B. If A[i-1] == B[j-1], dp[i][j] = dp[i-1][j-1] + 1; otherwise dp[i][j] = max(dp[i-1][j], dp[i][j-1]). This runs in O(n*m) time and space, where n and m are the string lengths.

Q68. Write a program to determine if a given number is prime, and optimize it for large inputs.

Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than sqrt(n), it must also have a corresponding factor smaller than sqrt(n)), skipping even numbers after checking 2, which reduces the check to roughly O(sqrt(n)). For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in O(n log log n), which is far more efficient than testing each number individually.

Q69. Write a program to merge overlapping intervals given a list of intervals.

Answer:

Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in O(n log n) time due to the initial sort.

Q70. Write a program to implement a queue using a fixed-size array (circular queue).

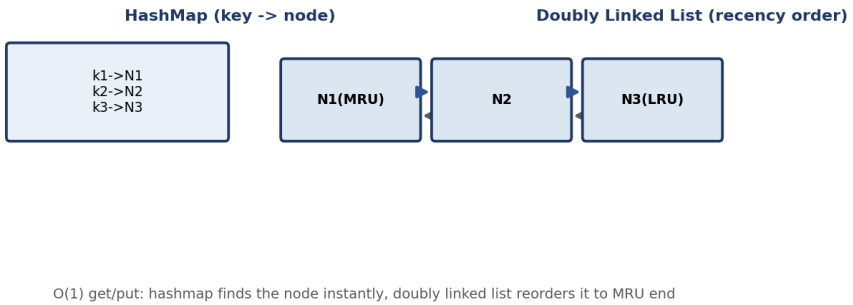
Answer:

A circular queue uses a fixed-size array with 'front' and 'rear' indices that wrap around using modulo arithmetic (rear = (rear + 1) % capacity) once they reach the end of the array, allowing the queue to reuse freed space at the beginning after elements have been dequeued - avoiding the wasted space of a naive fixed-array queue that never reuses earlier slots.

Q71. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On get(key): if present, move that node to the front (most recently used end) and return its value, else return -1/None. On put(key, value): if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in O(1).



Q72. Write a program to implement binary search on a rotated sorted array.

Answer:

Modify binary search by first determining which half of the array (from mid to low, or mid to high) is properly sorted, then checking if the target lies within that sorted half's range - if so, search recurse/iterate into that half, otherwise search the other half. This preserves the O(log n) time complexity of standard binary search despite the rotation.

Q73. Write a program to find all pairs in an array that sum up to a given target value.

Answer:

Use a hashmap: iterate through the array once, and for each element x, check if (target - x) already exists in the hashmap - if so, record the pair; otherwise, add x to the hashmap and continue. This finds all pairs summing to the target in O(n) time and O(n) space, instead of the O(n^2) brute-force nested loop.

Q74. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining n-k elements - this three-step reversal rotates the array by k positions in-place in O(n) time and O(1) extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading O(n) space for simpler code.)

Programming Language Fundamentals

Q75. What is duck typing, and how does it relate to Python's dynamic type system?

Answer:

Duck typing means an object's suitability for an operation is determined by whether it has the necessary methods/attributes ('if it walks like a duck and quacks like a duck...') rather than its explicit declared type. Python's dynamic typing supports this naturally - a function can accept any object as long as it implements the methods the function actually calls, without requiring a shared base class or interface.

Q76. What is a memory leak, and how would you identify one in a long-running application?

Answer:

A memory leak occurs when a program allocates memory but never releases it even though it's no longer needed, causing memory usage to grow over time and potentially exhausting available memory. In a long-running application, you'd identify one using a memory profiler to take heap snapshots over time and look for object counts/memory usage that keep growing without bound, then trace those objects back to what's still holding references to them (e.g. an ever-growing cache or a forgotten event-listener registration).

Q77. What is the difference between '==' and '.equals()' when comparing objects in Java?

Answer:

'==' compares object references - whether two variables point to the exact same object in memory. '.equals()' compares the logical content/value of two objects, based on however the class has overridden it (e.g. String's .equals() compares character sequences). Two distinct String objects with the same text will be '==' false but '.equals()' true, unless string interning makes them the same reference.

Q78. What is garbage collection, and how does it help prevent memory leaks?

Answer:

Garbage collection is an automatic memory management process that identifies objects no longer reachable/referenced by the running program and reclaims their memory, so the developer doesn't have to manually free it. This helps prevent memory leaks caused by forgetting to deallocate memory, though a program can still 'leak' memory logically if it keeps unnecessary references alive (e.g. objects sitting in a growing cache that's never cleared).

Q79. Explain Python's Global Interpreter Lock (GIL) and its impact on multithreaded CPU-bound programs.

Answer:

CPython's Global Interpreter Lock ensures only one thread executes Python bytecode at any given instant, even on a multi-core machine, to simplify memory management (reference counting) and avoid certain race conditions internally. This means CPU-bound multithreaded Python code doesn't actually get faster from adding more threads (they contend for the GIL) - for true parallel CPU-bound work, developers typically use multiprocessing (separate processes, each with its own interpreter and GIL) instead.

Q80. Explain the difference between a shallow copy and a deep copy of a nested data structure.

Answer:

A shallow copy creates a new outer object but copies references to the same nested objects as the original - mutating a nested list inside a shallow copy also affects the original. A deep copy recursively creates independent copies of every nested object, so the copy is fully isolated from the original at every level, at the cost of more time and memory to perform the copy.

Q81. Explain the difference between checked and unchecked exceptions in Java.

Answer:

Checked exceptions (e.g. IOException) are checked by the compiler at compile time - a method must either handle them with try-catch or declare them with 'throws', forcing the caller to acknowledge the possibility of failure. Unchecked exceptions (e.g. NullPointerException, extending RuntimeException) are not checked at compile time and don't require explicit handling, typically representing programming errors that could occur almost anywhere.

Q82. Explain the difference between pass-by-value and pass-by-reference with an example.

Answer:

Pass-by-value copies the actual value of a variable into the function's parameter, so changes inside the function don't affect the original variable. Pass-by-reference passes a reference to the original variable's memory location, so changes inside the function do affect the original. Example: in a language with true pass-by-reference, swap(a, b) can actually swap the caller's variables; with pass-by-value, it would only swap local copies, leaving the caller's variables unchanged.

Web & Cloud Fundamentals

Q83. What is auto-scaling in cloud computing, and what metrics typically trigger it?

Answer:

Auto-scaling automatically adjusts the number of running compute instances up or down based on real-time demand, rather than requiring manual intervention. Common triggering metrics include CPU utilization, memory usage, request/queue length, and response latency - when a threshold is crossed for a sustained period, the auto-scaler adds or removes instances accordingly.

Q84. What is the difference between HTTP and HTTPS, and how is HTTPS implemented under the hood?

Answer:

HTTP transmits data in plaintext, so it's vulnerable to eavesdropping and tampering in transit. HTTPS secures HTTP by layering it on top of TLS/SSL: a handshake negotiates a shared session key and verifies the server's identity via its certificate, and all subsequent HTTP traffic is encrypted using that session key, protecting confidentiality and integrity even over an untrusted network.

Q85. What is a RESTful API, and what makes an API design 'RESTful'?

Answer:

A RESTful API is an API designed around REST (Representational State Transfer) architectural principles: it uses standard HTTP methods (GET, POST, PUT, DELETE) mapped to resource operations, is stateless (each request contains all information needed), uses resource-based URLs (e.g. /users/123), and typically returns representations of resources (often JSON). An API is 'RESTful' to the extent it actually follows these constraints rather than just using HTTP as a transport.

Q86. What is containerization (e.g., Docker), and how is it different from a virtual machine?

Answer:

Containerization packages an application with its dependencies and runtime into a lightweight, portable container (e.g. via Docker) that shares the host OS kernel, making containers fast to start and resource-efficient. A virtual machine, by contrast, virtualizes an entire hardware stack including its own full guest OS, making VMs heavier and slower to start but providing stronger isolation between workloads.

Q87. Explain the difference between authentication and authorization, and how OAuth 2.0 fits in.

Answer:

Authentication verifies who a user is (e.g. checking a username/password or token), while authorization determines what an authenticated user is allowed to do (e.g. which resources/actions they can access). OAuth 2.0 is an authorization framework that lets a user grant a third-party application limited access to their resources on another service (e.g. 'Sign in with Google') without sharing their actual password, typically issuing access tokens that represent specific granted permissions.

Q88. Explain how a message queue like Kafka or RabbitMQ helps decouple microservices.

Answer:

A message queue like Kafka or RabbitMQ sits between producer services (that generate events/messages) and consumer services (that process them), letting producers publish messages without waiting for consumers to be ready or fast. This decouples services in time and load - producers and consumers can scale, fail, or be deployed independently - and provides buffering so a burst of traffic doesn't overwhelm downstream consumers directly.

Q89. Explain the difference between IaaS, PaaS, and SaaS with real examples of each.

Answer:

IaaS (Infrastructure as a Service) provides raw virtualized compute/storage/networking that you configure yourself, e.g. AWS EC2 or Azure VMs. PaaS (Platform as a Service) provides a managed platform to build and deploy applications without managing the underlying servers, e.g. Heroku or Azure App Service. SaaS (Software as a Service) delivers a complete, ready-to-use application over the internet, e.g. Gmail or Salesforce - you just use it, with no infrastructure or platform management at all.

Q90. Explain how a CDN improves website performance for geographically distributed users.

Answer:

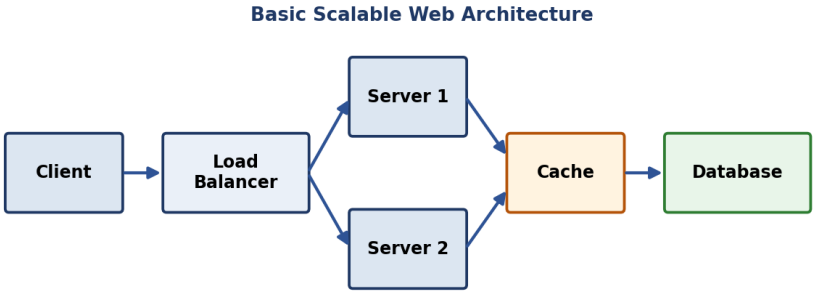
A CDN caches static content (images, CSS, JS, video) on edge servers distributed across many geographic locations. When a user requests that content, it's served from the nearest edge server instead of traveling all the way to the origin server, reducing round-trip latency, offloading traffic from the origin, and improving load times especially for users far from the origin's data center.

System Design Basics

Q91. How would you design a URL shortening service like bit.ly? Discuss the key components.

Answer:

Key components: a hash/encoding function (e.g. base62 encoding of an auto-incrementing ID, or a hash of the URL) to generate a short unique code; a database (often a fast key-value store) mapping short codes to original URLs; a redirect service that looks up the short code and issues an HTTP redirect; and a cache (e.g. Redis) in front of the database to serve frequently accessed short URLs quickly, since read traffic (redirects) vastly outnumbers write traffic (new URL creation).



Q92. Explain the trade-offs between strong consistency and eventual consistency in a distributed system.

Answer:

Strong consistency guarantees that once a write completes, every subsequent read (from any replica) sees that write immediately - simpler to reason about, but often requires more coordination between nodes, increasing latency and reducing availability during network issues. Eventual consistency allows replicas to temporarily diverge after a write, guaranteeing only that all replicas will converge to the same value if no new writes occur for some time - offering better availability and lower latency, at the cost of applications occasionally reading stale data.

Q93. Explain how you would design a notification system that can send millions of push notifications reliably.

Answer:

A reliable large-scale notification system typically uses a message queue (e.g. Kafka) to decouple the event that triggers a notification from the actual delivery: the triggering service publishes an event to the queue, and a pool of worker consumers pulls from the queue and calls the relevant push notification provider (e.g. FCM/APNs) for each recipient, retrying failed deliveries with backoff. This design absorbs traffic spikes, allows independent scaling of ingestion vs delivery workers, and avoids losing notifications if a downstream provider is temporarily slow or unavailable.

Q94. Explain the CAP theorem and how it applies to designing a distributed database.

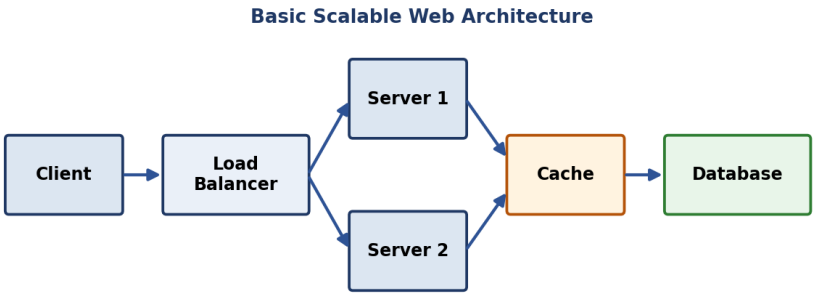
Answer:

The CAP theorem states that a distributed system can only guarantee two of the following three properties at the same time during a network partition: Consistency (every read gets the latest write), Availability (every request gets a non-error response), and Partition tolerance (the system keeps working despite network failures between nodes). Since partitions are essentially unavoidable in real distributed systems, designers effectively choose between prioritizing consistency (CP systems, e.g. many traditional databases in cluster mode) or availability (AP systems, e.g. many NoSQL databases) when a partition occurs.

Q95. What is a load balancer, and what are common load-balancing strategies (round robin, least connections, etc.)?

Answer:

A load balancer distributes incoming client requests across multiple backend servers to prevent any single server from being overwhelmed and to improve availability if a server goes down. Common strategies: Round Robin (cycles requests evenly across servers), Least Connections (routes to the server currently handling the fewest active connections), Weighted Round Robin (accounts for servers with different capacities), and IP Hash (routes based on client IP for session stickiness).



Q96. What is idempotency, and why is it important when designing payment or retry-safe APIs?

Answer:

Idempotency means performing an operation multiple times has the same effect as performing it once. It's crucial for payment or retry-safe APIs because network failures often make clients unsure whether a request actually succeeded, causing them to retry - without idempotency, a retried payment request could charge a customer twice. It's typically implemented by having the client generate a unique idempotency key per logical operation, which the server stores and checks against on each retry to avoid re-processing the same request.

Q97. How would you design a rate limiter for an API?

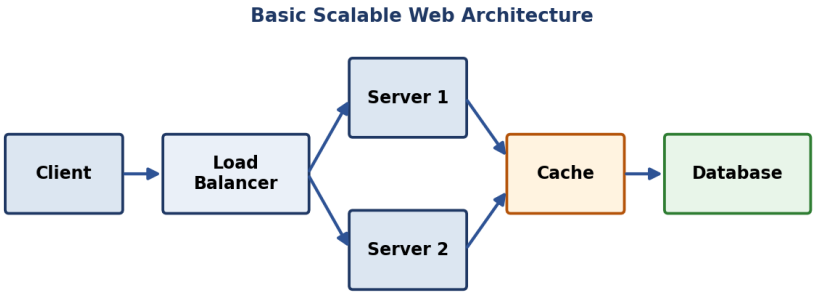
Answer:

A common design uses the 'token bucket' or 'sliding window' algorithm: each client is allocated a bucket of tokens that refills at a fixed rate, and each request consumes a token - requests are rejected (or queued) once the bucket is empty, until it refills. This state (token counts and timestamps) is typically stored in a fast shared store like Redis so the rate limiter works consistently across multiple servers handling the same client's requests.

Q98. What is the difference between horizontal and vertical scaling, and when would you use each?

Answer:

Horizontal scaling adds more machines/servers to distribute load across them (scale out), which is generally more resilient (no single point of failure) and has no hard upper limit, but requires the application to be designed to run across multiple nodes (e.g. stateless services, distributed data). Vertical scaling increases the resources (CPU/RAM) of a single existing machine (scale up), which is simpler with no architecture changes needed, but is limited by the maximum capacity of a single machine and doesn't improve fault tolerance.



Q99. How would you design a basic caching layer for a high-traffic read-heavy application?

Answer:

For a high-traffic, read-heavy application, place a cache (e.g. Redis or Memcached) between the application servers and the database. On a read, check the cache first (a 'cache hit' returns data instantly without touching the database); on a miss, read from the database and populate the cache for next time. Choose a cache invalidation/expiration strategy (TTL, or explicit invalidation on writes) to balance freshness against cache-hit rate, and consider a cache-aside or write-through pattern depending on how tolerant the application is of slightly stale reads.

Q100. What is database sharding, and what challenges does it introduce (e.g., cross-shard queries)?

Answer:

Database sharding splits a large table's data horizontally across multiple database servers based on a shard key, so each shard holds only a subset of rows, improving write throughput and storage scalability beyond what one machine can handle. Challenges include queries or transactions that need data spanning multiple shards (requiring expensive cross-shard joins or scatter-gather queries), rebalancing data when adding/removing shards, and maintaining a consistent, well-distributed choice of shard key to avoid 'hot' shards.

Note: 24 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.